CANDIDATE SCREENING / TECHNICAL CODING TEST

Next.js / TypeScript

Mid-Level Framework & Component Coding Assessment — Completed Solution

5 CODING CHALLENGES · 100 POINTS

Instructions

Use TypeScript with React and the Next.js App Router where appropriate. All challenges must be implemented within a single Next.js application, with a user interface (UI) to test and verify the functionality of each solution. For challenges that require implementing UI components, component structure, UI design, and styling will also be evaluated. Follow the stated requirements and examples. Prefer correct, accessible, maintainable code and handle the specified edge cases. Each problem is worth 20 points.

How this submission follows the instructions

Instruction	How it is satisfied
TypeScript + React + App Router	Next.js 15 (App Router, `src/app`), React 19, TypeScript in `strict` mode with `noUncheckedIndexedAccess`.
Single Next.js application with UI to test each solution	One app, routes `/challenge-1` … `/challenge-5`, each with an interactive playground; a home page links to all of them.
Component structure, UI design and styling evaluated	Components live in `src/components/*` with co-located CSS Modules (built into Next.js, no extra dependency), design tokens, responsive grid, visible focus states, disabled/out-of-stock styling.
Correct, accessible, maintainable code	Pure logic isolated in `src/lib`, documented, side-effect free; semantic HTML and WAI-ARIA patterns for the UI components.
Handle the specified edge cases	Every edge case in the statements is covered by an automated test (26 tests, all passing).

Verification summary

- `tsc --noEmit` (strict): no errors.
- `next build`: compiled successfully; all routes generated.
- `npm test` (node:test via tsx): 26 / 26 passing, including a 100,000-product performance test (about 150 ms) and a 10,000-pattern route test.
- Rendered HTML checked for the ProductCard (semantic markup, `Rp 12.500`), Tabs (roles and ARIA ids) and the live breadcrumb on `/products/blue%20mug`.

Project structure

```
nextjs-screening-assessment/
├─ package.json · tsconfig.json · next.config.ts
└─ src/
   ├─ lib/                      # pure, framework-free logic (unit tested)
   │  ├─ searchProducts.ts      # Challenge 1
   │  ├─ findRoute.ts           # Challenge 2
   │  ├─ buildBreadcrumbs.ts    # Challenge 3
   │  ├─ formatIDR.ts           # Challenge 4 helper
   │  └─ *.test.ts
   ├─ components/
   │  ├─ Breadcrumbs/           # Challenge 3 UI (+ live SiteBreadcrumbs)
   │  ├─ ProductCard/           # Challenge 4
   │  └─ Tabs/                  # Challenge 5
   └─ app/                      # App Router: layout, home, /challenge-1..5, /products
```

Run it

```
npm install
npm run dev      # http://localhost:3000
npm test           # unit tests
npm run typecheck && npm run build
```

1. Filter, Sort, and Paginate Products

TYPESCRIPT · DATA TRANSFORMATION · 20 POINTS

Problem

Implement a pure function for a product listing. Match product names containing the query (case-insensitive), sort matching names in ascending case-insensitive order, then return the requested one-based page. Treat page values below 1 as 1. If pageSize is below 1, return an empty array. Do not mutate the input.

```
type Product = { id: number; name: string };

function searchProducts(products: Product[], query: string, page: number, pageSize: number): Product[]
```

Examples. Constraints: 0 ≤ products.length ≤ 100,000; names and query may contain mixed case. Example 1: [{id:2,name:"Pear"},{id:1,name:"Apple"},{id:3,name:"Apricot"}], query "ap", page 1, pageSize 1 → [{id:1,name:"Apple"}]. Example 2: [{id:1,name:"Apple"}], query "zz", page 1, pageSize 10 → [].

Submission requirements. Return only items on the requested page. Target O(n log n) time or better.

Approach and best practices

- Single linear pass filters and pre-computes the lower-cased key once per product, so the comparator does no string allocation (the naive `toLowerCase()` inside `sort` would do it O(n log n) times).
- Total cost is O(n) filter + O(m log m) sort + O(pageSize) slice, which meets the O(n log n) target; only the page is returned.
- Never mutates: filtering builds a new array and sorting happens on that copy. Same item objects are returned, not clones.
- Plain code-unit comparison of lower-cased keys is deterministic and locale-independent; `Array.prototype.sort` is stable (ES2019), so equal names keep input order.
- Defensive input handling: page < 1 or non-finite → 1; pageSize < 1 or non-finite → `[]`; fractional values are floored.

Solution

src/lib/searchProducts.ts

```
export type Product = { id: number; name: string };

/**
 * Filter products by name (case-insensitive substring), sort the matches by
 * name (case-insensitive, ascending) and return the requested 1-based page.
 *
 * - Pure: the input array and its items are never mutated.
 * - page < 1 (or not a finite number) is treated as 1.
 * - pageSize < 1 (or not a finite number) returns [].
 * - Complexity: O(n) filter + O(m log m) sort (m = matches), so O(n log n).
 *   Lower-cased keys are computed once per item instead of inside the
 *   comparator, which keeps the sort fast for 100,000 items.
 * - Sort is stable (ES2019), so equal names keep their original order.
 */
export function searchProducts(
  products: Product[],
  query: string,
  page: number,
  pageSize: number,
): Product[] {
  if (!Number.isFinite(pageSize) || pageSize < 1) return [];

  const size = Math.floor(pageSize);
```

```
  const safePage = Number.isFinite(page) ? Math.max(1, Math.floor(page)) : 1;
  const needle = query.toLowerCase();

  const matches: { key: string; product: Product }[] = [];
  for (const product of products) {
    const key = product.name.toLowerCase();
    if (key.includes(needle)) matches.push({ key, product });
  }

  matches.sort((a, b) => (a.key < b.key ? -1 : a.key > b.key ? 1 : 0));

  const start = (safePage - 1) * size;
  return matches.slice(start, start + size).map((m) => m.product);
}
```

Edge cases covered

Spec examples, case-insensitive match and sort, page ≤ 0, pageSize ≤ 0, page past the end, empty query, empty input, input immutability, and a 100,000-item performance test.

UI to test it — src/app/challenge-1/SearchPlayground.tsx

```
"use client";

import { useId, useMemo, useState } from "react";
import { searchProducts, type Product } from "@/lib/searchProducts";

const WORDS = ["Apple", "apricot", "Pear", "banana", "Mango", "Papaya", "grape", "Cherry", "Melon", "Lime"];
const DATA: Product[] = Array.from({ length: 20_000 }, (_, i) => ({
  id: i + 1,
  name: `${WORDS[i % WORDS.length]} ${(i * 7919) % 20_000}`,
}));

export function SearchPlayground() {
  const [query, setQuery] = useState("ap");
  const [page, setPage] = useState(1);
  const [pageSize, setPageSize] = useState(10);
  const ids = { q: useId(), p: useId(), s: useId() };

  const results = useMemo(() => searchProducts(DATA, query, page, pageSize), [query, page, pageSize]);

  return (
    <section className="panel-box" aria-label="Search playground">
      <div className="row">
        <label className="field" htmlFor={ids.q}>
          Query
          <input id={ids.q} value={query} onChange={(e) => setQuery(e.target.value)} />
        </label>
        <label className="field" htmlFor={ids.p}>
          Page
          <input id={ids.p} type="number" value={page} onChange={(e) => setPage(Number(e.target.value))} />
        </label>
        <label className="field" htmlFor={ids.s}>
          Page size
          <input id={ids.s} type="number" value={pageSize} onChange={(e) => setPageSize(Number(e.target.value))} />
        </label>
      </div>
      <p aria-live="polite">
        <span className={`badge ${results.length ? "ok" : "no"}`}>{results.length} result(s)</span>
      </p>
      <pre className="out" tabIndex={0} aria-label="Results as JSON">
        {JSON.stringify(results, null, 2)}
      </pre>
    </section>
  );
}
```

2. Resolve the Best Matching Route

APP ROUTER · ROUTE MATCHING · 20 POINTS

Problem

Implement route matching for an ordered list of App Router-style patterns. A segment beginning with : matches exactly one non-empty path segment; other segments match literally. Ignore one trailing slash on either path. Return the index of the first matching pattern, or -1. A match must consume every path segment.

```
function findRoute(patterns: string[], pathname: string): number
```

Examples. Constraints: 0 ≤ patterns.length ≤ 10,000; paths are absolute and contain no query string; path length ≤ 2,000. Example 1: ["/products/:id", "/about"], "/products/42" → 0. Example 2: ["/products/:id"], "/products/42/reviews" → -1. Example 3: ["/about"], "/about/" → 0.

Submission requirements. Return the first matching pattern index; do not treat a partial segment match as a full match.

Approach and best practices

- Segment-based comparison (not string prefix or regex), so `/about` never matches `/aboutus`, and there is no regex-injection or backtracking risk.
- Path is split once; each pattern is rejected immediately when its segment count differs, which also enforces "must consume every segment".
- One shared `toSegments` helper normalises patterns and paths identically (single trailing slash removed, root → zero segments).
- A `:param` only matches a non-empty segment, so `/p/` and `/p//` do not match `/p/:id`; only ONE trailing slash is ignored.

Solution

src/lib/findRoute.ts

```ts
/** Split an absolute path into segments, ignoring ONE trailing slash. */
function toSegments(path: string): string[] {
  let p = path;
  if (p.length > 1 && p.endsWith("/")) p = p.slice(0, -1);
  if (p.startsWith("/")) p = p.slice(1);
  return p === "" ? [] : p.split("/");
}

/**
 * Return the index of the first pattern that matches `pathname`, or -1.
 *
 * - `:name` matches exactly one NON-EMPTY segment; other segments are literal
 *   and must match the whole segment (no partial matches).
 * - A match must consume every segment of the path.
 * - One trailing slash is ignored on both the pattern and the path.
 * - The path is split once; each pattern is rejected early when its segment
 *   count differs, so the cost is O(P * S) in the worst case.
 */
export function findRoute(patterns: string[], pathname: string): number {
  const segments = toSegments(pathname);

  for (let i = 0; i < patterns.length; i++) {
    const pattern = toSegments(patterns[i] ?? "");
    if (pattern.length !== segments.length) continue;

    let matched = true;
    for (let j = 0; j < pattern.length; j++) {
      const p = pattern[j] as string;
      const s = segments[j] as string;
      const ok = p.startsWith(":") ? s !== "" : p === s;
      if (!ok) {
        matched = false;
        break;
      }
    }
```

```
    if (matched) return i;
  }
  return -1;
}
```

Edge cases covered

Spec examples, first-match precedence in both orders, partial segment names, empty dynamic segment, missing segment, root route, double trailing slash, empty pattern list, and 10,000 patterns.

UI to test it — src/app/challenge-2/RoutePlayground.tsx

```
"use client";

import { useId, useState } from "react";
import { findRoute } from "@/lib/findRoute";

export function RoutePlayground() {
  const [patterns, setPatterns] = useState("/products/:id\n/about\n/users/:id/posts/:postId");
  const [pathname, setPathname] = useState("/products/42");
  const ids = { a: useId(), b: useId() };

  const list = patterns.split("\n").map((l) => l.trim()).filter(Boolean);
  const index = findRoute(list, pathname);

  return (
    <section className="panel-box" aria-label="Route matching playground">
      <div className="row">
        <label className="field" htmlFor={ids.a}>
          Patterns
          <textarea id={ids.a} rows={5} value={patterns} onChange={(e) => setPatterns(e.target.value)} />
        </label>
        <label className="field" htmlFor={ids.b}>
          Pathname
          <input id={ids.b} value={pathname} onChange={(e) => setPathname(e.target.value)} />
        </label>
      </div>
      <p aria-live="polite">
        Result: <span className={`badge ${index >= 0 ? "ok" : "no"}`}>{index}</span>{" "}
        {index >= 0 ? <code>{list[index]}</code> : "no match"}
      </p>
    </section>
  );
}
```

3. Build a Safe Breadcrumb Trail

NEXT.JS ROUTING · STRINGS · 20 POINTS

Problem

Given a pathname and a map of known segment labels, return breadcrumb objects for every non-empty path prefix. Decode percent-encoded segments. Use the mapped label when available; otherwise use the decoded segment with hyphens replaced by spaces. Each href is the encoded URL prefix through that segment.

```
type Breadcrumb = { label: string; href: string };

function buildBreadcrumbs(pathname: string, labels: Record<string, string>): Breadcrumb[]
```

Examples. Constraints: pathname is absolute and may have a trailing slash; labels are keyed by decoded individual segment. Example 1: "/products/blue%20mug", {products:"Shop"} → [{label:"Shop",href:"/products"},{label:"blue mug",href:"/products/blue%20mug"}]. Example 2: "/", {} → [].

Submission requirements. Return breadcrumbs in path order and ensure each href is an encoded path prefix.

Approach and best practices

- "Safe" decoding: `decodeURIComponent` throws on malformed input such as `%E0%A4%A`; a `safeDecode` wrapper keeps the segment literal instead of crashing the page.
- hrefs are rebuilt with `encodeURIComponent` on the decoded segment, so they are always canonical and valid (spaces, &, unicode, an encoded `%2F` are all preserved correctly, with no double encoding).
- Labels are looked up with `hasOwnProperty`, so a URL segment like `constructor` or `__proto__` cannot pick up inherited properties of the labels object.
- Empty segments (leading, trailing or duplicate slashes) are skipped; the root path yields an empty array.
- The Breadcrumbs component is accessible: `<nav aria-label>`, an ordered list, and `aria-current="page"` on the last item. It is also wired to the real URL through `usePathname` in the root layout.

Solution

src/lib/buildBreadcrumbs.ts

```
export type Breadcrumb = { label: string; href: string };

/** decodeURIComponent that never throws on malformed input such as "%E0%A4%A". */
function safeDecode(segment: string): string {
  try {
    return decodeURIComponent(segment);
  } catch {
    return segment;
  }
}

/**
 * Build breadcrumbs for every non-empty prefix of `pathname`.
 *
 * - Segments are percent-decoded safely (malformed escapes are kept literally).
 * - Label: mapped label when the decoded segment is an OWN key of `labels`
 *   (so "constructor" / "__proto__" are not mistaken for labels), otherwise
 *   the decoded segment with hyphens replaced by spaces.
 * - href: the encoded prefix, re-encoded with encodeURIComponent so it is
 *   always a valid, canonical path ("blue mug" -> "blue%20mug", "a/b" -> "a%2Fb").
 */
export function buildBreadcrumbs(
  pathname: string,
  labels: Record<string, string>,
): Breadcrumb[] {
  const crumbs: Breadcrumb[] = [];
  let href = "";
```

```
  for (const raw of pathname.split("/")) {
    if (raw === "") continue;

    const decoded = safeDecode(raw);
    href += `/${encodeURIComponent(decoded)}`;

    const label = Object.prototype.hasOwnProperty.call(labels, decoded)
      ? (labels[decoded] as string)
      : decoded.replace(/-/g, " ");

    crumbs.push({ label, href });
  }
  return crumbs;
}
```

src/components/Breadcrumbs/Breadcrumbs.tsx

```
import type { JSX } from "react";
import Link from "next/link";
import { buildBreadcrumbs } from "@/lib/buildBreadcrumbs";
import styles from "./Breadcrumbs.module.css";

type BreadcrumbsProps = {
  pathname: string;
  labels: Record<string, string>;
};

/** Presentational breadcrumb <nav>; the trail itself comes from buildBreadcrumbs. */
export function Breadcrumbs({ pathname, labels }: BreadcrumbsProps): JSX.Element | null {
  const crumbs = buildBreadcrumbs(pathname, labels);
  if (crumbs.length === 0) return null;

  return (
    <nav aria-label="Breadcrumb" className={styles.nav}>
      <ol className={styles.list}>
        <li className={styles.item}>
          <Link href="/">Home</Link>
        </li>
        {crumbs.map((crumb, i) => {
          const isLast = i === crumbs.length - 1;
          return (
            <li key={crumb.href} className={styles.item}>
              {isLast ? (
                <span aria-current="page">{crumb.label}</span>
              ) : (
                <Link href={crumb.href}>{crumb.label}</Link>
              )}
            </li>
          );
        })}
      </ol>
    </nav>
  );
}
```

src/components/Breadcrumbs/SiteBreadcrumbs.tsx

```
"use client";

import type { JSX } from "react";
import { usePathname } from "next/navigation";
import { Breadcrumbs } from "./Breadcrumbs";

const LABELS: Record<string, string> = {
  products: "Shop",
  "challenge-1": "Challenge 1 · Search products",
  "challenge-2": "Challenge 2 · Route matching",
  "challenge-3": "Challenge 3 · Breadcrumbs",
  "challenge-4": "Challenge 4 · Product card",
  "challenge-5": "Challenge 5 · Tabs",
};

/** Live breadcrumb for the whole app, driven by the real URL. */
export function SiteBreadcrumbs(): JSX.Element | null {
  const pathname = usePathname();
  return <Breadcrumbs pathname={pathname} labels={LABELS} />;
}
```

src/components/Breadcrumbs/Breadcrumbs.module.css

```
.list {
  display: flex;
  flex-wrap: wrap;
  gap: 0.4rem;
  padding: 0;
  margin: 0;
  font-size: 0.9rem;
  list-style: none;
}

.item + .item::before {
  margin-right: 0.4rem;
  color: var(--text-muted);
  content: "/";
}

.item a {
  color: var(--accent-strong);
  text-decoration: none;
}

.item a:hover {
  text-decoration: underline;
}

.item a:focus-visible {
  outline: 3px solid var(--focus);
  outline-offset: 2px;
  border-radius: 4px;
}

.item [aria-current="page"] {
  font-weight: 600;
  color: var(--text);
}
```

Edge cases covered

Spec examples, trailing and duplicate slashes, hyphen replacement only for unmapped segments, labels keyed by the decoded segment, unicode and reserved characters, malformed percent-encoding, and prototype-key names.

UI to test it — src/app/challenge-3/BreadcrumbPlayground.tsx

```
"use client";

import { useId, useState } from "react";
import { buildBreadcrumbs } from "@/lib/buildBreadcrumbs";

export function BreadcrumbPlayground() {
  const [pathname, setPathname] = useState("/products/blue%20mug");
  const [labelsText, setLabelsText] = useState('{ "products": "Shop" }');
  const ids = { a: useId(), b: useId() };

  let labels: Record<string, string> = {};
  let error: string | null = null;
  try {
    const parsed: unknown = JSON.parse(labelsText);
    if (parsed && typeof parsed === "object" && !Array.isArray(parsed)) {
      labels = parsed as Record<string, string>;
    } else {
      error = "Labels must be a JSON object.";
    }
  } catch {
    error = "Labels is not valid JSON.";
  }

  return (
    <section className="panel-box" aria-label="Breadcrumb playground">
      <div className="row">
        <label className="field" htmlFor={ids.a}>
          Pathname
          <input id={ids.a} value={pathname} onChange={(e) => setPathname(e.target.value)} />
        </label>
        <label className="field" htmlFor={ids.b}>
```

```
          Labels (JSON)
          <textarea id={ids.b} rows={3} value={labelsText} onChange={(e) => setLabelsText(e.target.value)} />
        </label>
      </div>
      {error && <p role="alert" className="badge no">{error}</p>}
      <pre className="out" tabIndex={0} aria-label="Breadcrumbs as JSON">
        {JSON.stringify(buildBreadcrumbs(pathname, labels), null, 2)}
      </pre>
    </section>
  );
}
```

4. Build a Product Card Component

REACT / NEXT.JS · COMPONENT IMPLEMENTATION · 20 POINTS

Problem

Build a reusable product card as a React component using TypeScript and the Next.js App Router. Display the product image, name, price formatted as IDR, and an “Add to cart” button. Clicking the button calls onAdd with the product id. If unavailable, show “Out of stock” and disable the button.

Use semantic HTML and accessible image text. Keep the component reusable and typed. Do not add data fetching or external dependencies.

```
type Product = { id: string; name: string; price: number; imageUrl: string; available: boolean };
type ProductCardProps = { product: Product; onAdd: (productId: string) => void };

export function ProductCard(props: ProductCardProps): JSX.Element
```

Examples. price 12500 displays as “Rp 12.500”; clicking the button calls onAdd(product.id). When available is false, the disabled button reads “Out of stock”.

Submission requirements. Submit the complete component implementation and any small currency-formatting helper.

Approach and best practices

- `"use client"` is required because the card handles clicks and receives a function prop (functions cannot be passed from a Server Component). The directive is kept on this leaf component only.
- Semantic markup: <article> labelled by its <h3> (via useId, SSR-safe), <data value> for the price, a real <button type="button">.
- Image has meaningful `alt` text (the product name), explicit width and height to avoid layout shift, and lazy and async loading. A plain <img> is used on purpose: `next/image` needs remote host configuration, which the task forbids adding.
- The button keeps its visible name (“Add to cart” / “Out of stock”) so it satisfies WCAG “label in name”, and is tied to the product with `aria-describedby`. The disabled state is a real `disabled` attribute plus a guard in the handler.
- `formatIDR` uses `Intl.NumberFormat('id-ID')` for the digits and adds the “Rp ” prefix itself, avoiding the non-breaking space the currency style inserts, so the output matches “Rp 12.500” exactly. It rounds to whole rupiah and handles negatives and NaN.
- Styling uses CSS Modules and design tokens: hover lift, visible `:focus-visible` ring, greyed image and disabled button for out-of-stock. No data fetching or third-party libraries.

Solution

src/lib/formatIDR.ts

```
const idNumber = new Intl.NumberFormat("id-ID", {
  maximumFractionDigits: 0,
});

/** 12500 -> "Rp 12.500". Rounds to whole rupiah; negatives -> "-Rp 1.000". */
export function formatIDR(amount: number): string {
  if (!Number.isFinite(amount)) return "Rp -";
  const rounded = Math.round(amount);
  const body = idNumber.format(Math.abs(rounded));
  return `${rounded < 0 ? "-" : ""}Rp ${body}`;
}
```

src/components/ProductCard/ProductCard.tsx

```
"use client";

import { useId } from "react";
import type { JSX } from "react";
import { formatIDR } from "@/lib/formatIDR";
import styles from "./ProductCard.module.css";

export type Product = {
```

```
  id: string;
  name: string;
  price: number;
  imageUrl: string;
  available: boolean;
};

export type ProductCardProps = {
  product: Product;
  onAdd: (productId: string) => void;
};

/**
 * Reusable, presentational product card.
 * - Client Component only because it handles a click and receives a function prop.
 * - Semantic markup: <article> labelled by its <h3>, <data> for the price.
 * - The button keeps a stable, visible accessible name ("Add to cart") and is
 *   linked to the product name through aria-describedby.
 */
export function ProductCard({ product, onAdd }: ProductCardProps): JSX.Element {
  const titleId = useId();
  const { id, name, price, imageUrl, available } = product;

  return (
    <article className={styles.card} aria-labelledby={titleId} data-available={available}>
      <div className={styles.media}>
        {/* Plain <img>: product images come from arbitrary hosts and the task
            forbids extra config/dependencies (next/image needs remotePatterns). */}
        {/* eslint-disable-next-line @next/next/no-img-element */}
        <img
          className={styles.image}
          src={imageUrl}
          alt={name}
          width={400}
          height={300}
          loading="lazy"
          decoding="async"
        />
      </div>

      <div className={styles.body}>
        <h3 id={titleId} className={styles.name}>
          {name}
        </h3>
        <p className={styles.price}>
          <data value={price}>{formatIDR(price)}</data>
        </p>

        <button
          type="button"
          className={styles.button}
          disabled={!available}
          aria-describedby={titleId}
          onClick={() => {
            if (available) onAdd(id);
          }}
        >
          {available ? "Add to cart" : "Out of stock"}
        </button>
      </div>
    </article>
  );
}
```

src/components/ProductCard/ProductCard.module.css

```
.card {
  display: flex;
  flex-direction: column;
  overflow: hidden;
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 14px;
  box-shadow: 0 1px 2px rgb(15 23 42 / 6%);
  transition: box-shadow 0.15s ease, transform 0.15s ease;
}
```

```css
.card:hover {
  box-shadow: 0 8px 24px rgb(15 23 42 / 10%);
  transform: translateY(-2px);
}

.media {
  aspect-ratio: 4 / 3;
  background: var(--surface-muted);
}

.image {
  display: block;
  width: 100%;
  height: 100%;
  object-fit: cover;
}

.card[data-available="false"] .image {
  filter: grayscale(1);
  opacity: 0.55;
}

.body {
  display: flex;
  flex: 1;
  flex-direction: column;
  gap: 0.5rem;
  padding: 1rem;
}

.name {
  margin: 0;
  font-size: 1rem;
  font-weight: 600;
  line-height: 1.35;
}

.price {
  margin: 0;
  font-size: 1.125rem;
  font-weight: 700;
  color: var(--accent-strong);
}

.button {
  margin-top: auto;
  padding: 0.65rem 1rem;
  font: inherit;
  font-weight: 600;
  color: #fff;
  cursor: pointer;
  background: var(--accent);
  border: 0;
  border-radius: 10px;
  transition: background 0.15s ease;
}

.button:hover:not(:disabled) {
  background: var(--accent-strong);
}

.button:focus-visible {
  outline: 3px solid var(--focus);
  outline-offset: 2px;
}

.button:disabled {
  color: var(--text-muted);
  cursor: not-allowed;
  background: var(--surface-muted);
}
```

Edge cases covered

12.500 / 0 / 999 / 1.234.567 formatting, rounding, negative and NaN amounts (unit tested); available vs unavailable rendering, disabled state, and the click callback with the product id (verified in the playground).

UI to test it — src/app/challenge-4/ProductGrid.tsx

```
"use client";

import { useState } from "react";
import { ProductCard, type Product } from "@/components/ProductCard/ProductCard";

/** Inline SVG placeholder so the demo needs no network or remote image config. */
const img = (color: string, label: string) =>
  `data:image/svg+xml;utf8,${encodeURIComponent(
    `<svg xmlns="http://www.w3.org/2000/svg" width="400" height="300"><rect width="400" height="300"
        fill="${color}"/><text x="200" y="165" font-size="44" text-anchor="middle" fill="#fff" font-family="sans-
        serif">${label}</text></svg>`,
  )}`;

const PRODUCTS: Product[] = [
  { id: "p1", name: "Blue Ceramic Mug", price: 12500, imageUrl: img("#2f5bea", "Mug"), available: true },
  { id: "p2", name: "Handmade Coffee Beans 250g", price: 89000, imageUrl: img("#7c4a21", "Coffee"), available: true },
  { id: "p3", name: "Limited Edition Tote Bag", price: 1250000, imageUrl: img("#0d9488", "Tote"), available: false },
];

export function ProductGrid() {
  const [cart, setCart] = useState<Record<string, number>>({});
  const total = Object.values(cart).reduce((a, b) => a + b, 0);

  return (
    <>
      <p aria-live="polite" className="panel-box">
        Items in cart: <strong>{total}</strong>
        {total > 0 && (
          <> · last: <code>{JSON.stringify(cart)}</code></>
        )}
      </p>
      <ul className="grid">
        {PRODUCTS.map((p) => (
          <li key={p.id}>
            <ProductCard
              product={p}
              onAdd={(id) => setCart((c) => ({ ...c, [id]: (c[id] ?? 0) + 1 }))}
            />
          </li>
        ))}
      </ul>
    </>
  );
}
```

5. Build an Accessible Tabs Component

REACT / NEXT.JS · COMPONENT IMPLEMENTATION · 20 POINTS

Problem

Build a reusable, controlled tabs component in React with TypeScript. It receives a list of tabs, active tab id, and onChange callback. Render one tab button per item and the active tab panel. Clicking a tab calls onChange with its id. Mark the selected tab accessibly and connect each tab to its panel with appropriate tab roles and ARIA attributes.

Assume tabs is non-empty and activeId matches one tab id. Use buttons that support keyboard focus. Do not use a UI component library.

```
type TabItem = { id: string; label: string; content: React.ReactNode };
type TabsProps = { tabs: TabItem[]; activeId: string; onChange: (id: string) => void };

export function Tabs(props: TabsProps): JSX.Element
```

Examples. when activeId is “reviews”, mark the Reviews tab as selected and show its content in the tab panel.

Submission requirements. Submit the complete component, including tab-list, tab-button, and active-panel markup.

Approach and best practices

- Follows the WAI-ARIA Authoring Practices “tabs with automatic activation” pattern: `role="tablist"`, `role="tab"` with `aria-selected`, and `role="tabpanel"`.
- Every tab has `aria-controls` pointing to its panel id and the panel has `aria-labelledby` pointing back to the active tab. DOM ids are index-based (`useId` + index), so arbitrary tab ids with spaces or symbols can never produce invalid ids.
- Roving tabindex: only the selected tab has `tabIndex=0` (one Tab stop for the whole list); the panel is focusable so keyboard users can reach its content.
- Keyboard support beyond the minimum: ← / → (wrapping), Home and End move focus and select the tab; handled through onKeyDown with `preventDefault`.
- Fully controlled: the component holds no selected-state of its own. It calls `onChange(id)` on click and key navigation, and only renders the active panel as specified. If `activeId` is ever invalid it falls back to the first tab instead of rendering nothing.
- Optional `ariaLabel` prop names the tab list for screen readers; no UI library is used.

Solution

src/components/Tabs/Tabs.tsx

```
"use client";

import { useId, useRef } from "react";
import type { JSX, KeyboardEvent, ReactNode } from "react";
import styles from "./Tabs.module.css";

export type TabItem = { id: string; label: string; content: ReactNode };

export type TabsProps = {
  tabs: TabItem[];
  activeId: string;
  onChange: (id: string) => void;
  /** Optional accessible name for the tab list. */
  ariaLabel?: string;
};

/**
 * Controlled, accessible tabs following the WAI-ARIA Authoring Practices
 * "Tabs with automatic activation" pattern:
 * - role=tablist / tab / tabpanel, aria-selected, aria-controls, aria-labelledby
 * - roving tabindex: only the selected tab is in the Tab order
 * - Arrow Left/Right (wrapping), Home and End move focus AND select the tab
 * - DOM ids are index based, so any user supplied tab id is safe
```

```
 */
export function Tabs({ tabs, activeId, onChange, ariaLabel }: TabsProps): JSX.Element {
  const baseId = useId();
  const buttonRefs = useRef<(HTMLButtonElement | null)[]>([]);

  const found = tabs.findIndex((t) => t.id === activeId);
  const activeIndex = found === -1 ? 0 : found; // defensive; spec guarantees a match
  const active = tabs[activeIndex];

  const tabDomId = (i: number) => `${baseId}-tab-${i}`;
  const panelDomId = (i: number) => `${baseId}-panel-${i}`;

  function select(index: number) {
    const next = tabs[index];
    if (!next) return;
    buttonRefs.current[index]?.focus();
    if (next.id !== activeId) onChange(next.id);
  }

  function handleKeyDown(event: KeyboardEvent<HTMLButtonElement>, index: number) {
    const last = tabs.length - 1;
    let target: number | null = null;

    switch (event.key) {
      case "ArrowRight":
        target = index === last ? 0 : index + 1;
        break;
      case "ArrowLeft":
        target = index === 0 ? last : index - 1;
        break;
      case "Home":
        target = 0;
        break;
      case "End":
        target = last;
        break;
      default:
        return;
    }
    event.preventDefault();
    select(target);
  }

  return (
    <div className={styles.tabs}>
      <div role="tablist" aria-label={ariaLabel} aria-orientation="horizontal" className={styles.list}>
        {tabs.map((tab, index) => {
          const selected = index === activeIndex;
          return (
            <button
              key={tab.id}
              ref={(el) => {
                buttonRefs.current[index] = el;
              }}
              type="button"
              role="tab"
              id={tabDomId(index)}
              aria-selected={selected}
              aria-controls={panelDomId(index)}
              tabIndex={selected ? 0 : -1}
              className={styles.tab}
              onClick={() => onChange(tab.id)}
              onKeyDown={(e) => handleKeyDown(e, index)}
            >
              {tab.label}
            </button>
          );
        })}
      </div>

      {active && (
        <div
          role="tabpanel"
          id={panelDomId(activeIndex)}
          aria-labelledby={tabDomId(activeIndex)}
          tabIndex={0}
          className={styles.panel}
```

```
        >
          {active.content}
        </div>
      )}
    </div>
  );
}
```

src/components/Tabs/Tabs.module.css

```
.list {
  display: flex;
  gap: 0.25rem;
  overflow-x: auto;
  border-bottom: 1px solid var(--border);
}

.tab {
  position: relative;
  padding: 0.75rem 1.1rem;
  font: inherit;
  font-weight: 600;
  color: var(--text-muted);
  white-space: nowrap;
  cursor: pointer;
  background: none;
  border: 0;
  border-radius: 8px 8px 0 0;
  transition: color 0.15s ease, background 0.15s ease;
}

.tab:hover {
  color: var(--text);
  background: var(--surface-muted);
}

.tab[aria-selected="true"] {
  color: var(--accent-strong);
}

.tab[aria-selected="true"]::after {
  position: absolute;
  right: 0;
  bottom: -1px;
  left: 0;
  height: 3px;
  content: "";
  background: var(--accent);
  border-radius: 3px 3px 0 0;
}

.tab:focus-visible,
.panel:focus-visible {
  outline: 3px solid var(--focus);
  outline-offset: -3px;
}

.panel {
  padding: 1.25rem 0.25rem;
  line-height: 1.6;
}
```

Edge cases covered

Selected state and tabindex for the active tab, one panel rendered, aria-controls ↔ aria-labelledby ids match, keyboard focus ring, wrap-around navigation, and an invalid activeId fallback (verified in the rendered HTML and the playground).

UI to test it — src/app/challenge-5/TabsDemo.tsx

```
"use client";

import { useState } from "react";
import { Tabs, type TabItem } from "@/components/Tabs/Tabs";

const TABS: TabItem[] = [
  { id: "description", label: "Description", content: <p>A sturdy ceramic mug that keeps drinks warm.</p> },
```

```
  { id: "reviews", label: "Reviews", content: <p>★★★★★ “Best mug I have owned.” — 128 reviews.</p> },
  { id: "shipping", label: "Shipping", content: <p>Ships within 2 business days across Indonesia.</p> },
];

export function TabsDemo() {
  const [activeId, setActiveId] = useState("reviews");
  return (
    <section className="panel-box">
      <Tabs tabs={TABS} activeId={activeId} onChange={setActiveId} ariaLabel="Product information" />
      <p className="lead" aria-live="polite">Active tab id: <code>{activeId}</code></p>
    </section>
  );
}
```

Appendix A — App shell and shared files

Layout, home page, global design tokens and the demo product routes.

src/app/layout.tsx

```
import type { Metadata } from "next";
import type { ReactNode } from "react";
import Link from "next/link";
import { SiteBreadcrumbs } from "@/components/Breadcrumbs/SiteBreadcrumbs";
import "./globals.css";

export const metadata: Metadata = {
  title: "Next.js Screening Assessment",
  description: "Five coding challenges implemented with Next.js App Router and TypeScript.",
};

const links = [
  { href: "/challenge-1", label: "1. Search products" },
  { href: "/challenge-2", label: "2. Route matching" },
  { href: "/challenge-3", label: "3. Breadcrumbs" },
  { href: "/challenge-4", label: "4. Product card" },
  { href: "/challenge-5", label: "5. Tabs" },
];

export default function RootLayout({ children }: { children: ReactNode }) {
  return (
    <html lang="en">
      <body>
        <header className="site-header">
          <div className="container">
            <Link href="/" style={{ fontWeight: 800, textDecoration: "none" }}>
              Next.js Screening Assessment
            </Link>
            <nav aria-label="Challenges">
              <ul className="site-nav">
                {links.map((l) => (
                  <li key={l.href}>
                    <Link href={l.href}>{l.label}</Link>
                  </li>
                ))}
              </ul>
            </nav>
          </div>
        </header>
        <main className="container">
          <SiteBreadcrumbs />
          {children}
        </main>
      </body>
    </html>
  );
}
```

src/app/page.tsx

```
import Link from "next/link";

const challenges = [
  { href: "/challenge-1", title: "1. Filter, sort and paginate products", note: "searchProducts() with a live
      playground over 20,000 items" },
  { href: "/challenge-2", title: "2. Resolve the best matching route", note: "findRoute() with editable patterns" },
  { href: "/challenge-3", title: "3. Safe breadcrumb trail", note: "buildBreadcrumbs() + the live breadcrumb above" },
  { href: "/challenge-4", title: "4. Product card component", note: "ProductCard with IDR formatting and out-of-stock
      state" },
  { href: "/challenge-5", title: "5. Accessible tabs component", note: "Controlled Tabs with full keyboard support" },
];

export default function HomePage() {
  return (
    <>
      <h1>Next.js / TypeScript screening</h1>
      <p className="lead">Five challenges, one app. Pick a challenge to test it.</p>
      <ul className="challenge-list">
        {challenges.map((c) => (
          <li key={c.href}>
            <Link href={c.href}>
```

```
              <strong>{c.title}</strong>
              <span>{c.note}</span>
            </Link>
          </li>
        ))}
      </ul>
    </>
  );
}
```

src/app/globals.css

```
:root {
  --bg: #f6f7fb;
  --surface: #fff;
  --surface-muted: #eceff5;
  --border: #dde1ea;
  --text: #1b2130;
  --text-muted: #5b6475;
  --accent: #2f5bea;
  --accent-strong: #1f3fb5;
  --focus: #f59e0b;
  color-scheme: light;
}

* { box-sizing: border-box; }

body {
  margin: 0;
  font-family: system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
  color: var(--text);
  background: var(--bg);
  line-height: 1.5;
}

a { color: var(--accent-strong); }

.container { max-width: 1040px; padding: 1.5rem 1rem 4rem; margin: 0 auto; }

.site-header { background: var(--surface); border-bottom: 1px solid var(--border); }
.site-header .container { padding-bottom: 1rem; }
.site-nav { display: flex; flex-wrap: wrap; gap: 0.25rem 1rem; padding: 0; margin: 0.5rem 0 0; list-style: none; }
.site-nav a { text-decoration: none; font-weight: 600; font-size: 0.95rem; }
.site-nav a:hover { text-decoration: underline; }

h1 { margin: 0 0 0.25rem; font-size: 1.75rem; }
.lead { margin: 0 0 1.5rem; color: var(--text-muted); }

.panel-box {
  padding: 1.25rem;
  margin-bottom: 1.25rem;
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 14px;
}

.field { display: flex; flex-direction: column; gap: 0.3rem; font-weight: 600; font-size: 0.9rem; }
.field input, .field textarea {
  padding: 0.55rem 0.7rem;
  font: inherit;
  font-weight: 400;
  color: var(--text);
  background: #fff;
  border: 1px solid var(--border);
  border-radius: 8px;
}
.field input:focus-visible, .field textarea:focus-visible, a:focus-visible {
  outline: 3px solid var(--focus);
  outline-offset: 1px;
}
.row { display: grid; grid-template-columns: repeat(auto-fit, minmax(160px, 1fr)); gap: 1rem; }

pre.out {
  padding: 0.9rem;
  margin: 0;
  overflow-x: auto;
  font-size: 0.85rem;
```

```
  background: #0f172a;
  color: #e2e8f0;
  border-radius: 10px;
}

.grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(220px, 1fr)); gap: 1.25rem; padding: 0; margin:
    0; list-style: none; }

.badge { display: inline-block; padding: 0.15rem 0.6rem; font-size: 0.85rem; font-weight: 700; border-radius: 999px; }
.badge.ok { color: #14532d; background: #dcfce7; }
.badge.no { color: #7f1d1d; background: #fee2e2; }

.sr-only { position: absolute; width: 1px; height: 1px; padding: 0; margin: -1px; overflow: hidden; clip: rect(0 0 0
   0); white-space: nowrap; border: 0; }

.challenge-list { display: grid; gap: 1rem; padding: 0; margin: 0; list-style: none; }
.challenge-list a { display: block; padding: 1rem 1.25rem; text-decoration: none; background: var(--surface); border:
   1px solid var(--border); border-radius: 14px; }
.challenge-list a:hover { border-color: var(--accent); }
.challenge-list strong { display: block; color: var(--text); }
.challenge-list span { color: var(--text-muted); font-size: 0.9rem; }
```

src/app/products/page.tsx

```
import Link from "next/link";

export default function ProductsPage() {
  return (
    <>
      <h1>Shop</h1>
      <p className="lead">
        Open <Link href="/products/blue%20mug">blue mug</Link> or{" "}
        <Link href="/products/red-tea-pot">red-tea-pot</Link>.
      </p>
    </>
  );
}
```

src/app/products/[slug]/page.tsx

```
type Props = { params: Promise<{ slug: string }> };

export default async function ProductPage({ params }: Props) {
  const { slug } = await params;
  const name = decodeURIComponent(slug).replace(/-/g, " ");
  return (
    <>
      <h1 style={{ textTransform: "capitalize" }}>{name}</h1>
      <p className="lead">Check the breadcrumb above: it is built from the real URL.</p>
    </>
  );
}
```

src/app/challenge-1/page.tsx

```
import type { Metadata } from "next";
import { SearchPlayground } from "./SearchPlayground";

export const metadata: Metadata = { title: "Challenge 1 · Search products" };

export default function Page() {
  return (
    <>
      <h1>Filter, sort and paginate products</h1>
      <p className="lead">Dataset: 20,000 generated products. Try query “ap”, page 0, page size 0.</p>
      <SearchPlayground />
    </>
  );
}
```

src/app/challenge-4/page.tsx

```
import type { Metadata } from "next";
import { ProductGrid } from "./ProductGrid";

export const metadata: Metadata = { title: "Challenge 4 · Product card" };

export default function Page() {
  return (
```

```
    <>
      <h1>Product card component</h1>
      <p className="lead">Click “Add to cart”. The out-of-stock card has a disabled button.</p>
      <ProductGrid />
    </>
  );
}
```

package.json

```
{
  "name": "nextjs-screening-assessment",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "typecheck": "tsc --noEmit",
    "test": "tsx --test \"src/**/*.test.ts\" \"src/**/*.test.tsx\""
  },
  "dependencies": {
    "next": "15.5.4",
    "react": "19.1.1",
    "react-dom": "19.1.1"
  },
  "devDependencies": {
    "@types/node": "^22.0.0",
    "@types/react": "^19.1.0",
    "@types/react-dom": "^19.1.0",
    "tsx": "^4.19.0",
    "typescript": "^5.6.0"
  }
}
```

tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": false,
    "skipLibCheck": true,
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true,
    "plugins": [{ "name": "next" }],
    "paths": { "@/*": ["./src/*"] }
  },
  "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx", ".next/types/**/*.ts"],
  "exclude": ["node_modules"]
}
```

next.config.ts

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  reactStrictMode: true,
};

export default nextConfig;
```

Appendix B — Unit tests (node:test)

Run with `npm test`. 26 tests, all passing.

src/lib/searchProducts.test.ts

```
import { test } from "node:test";
import assert from "node:assert/strict";
import { searchProducts, type Product } from "./searchProducts";

const sample: Product[] = [
  { id: 2, name: "Pear" },
  { id: 1, name: "Apple" },
  { id: 3, name: "Apricot" },
];

test("example 1: filter, sort, paginate", () => {
  assert.deepEqual(searchProducts(sample, "ap", 1, 1), [{ id: 1, name: "Apple" }]);
  assert.deepEqual(searchProducts(sample, "ap", 2, 1), [{ id: 3, name: "Apricot" }]);
});

test("example 2: no match", () => {
  assert.deepEqual(searchProducts([{ id: 1, name: "Apple" }], "zz", 1, 10), []);
});

test("case-insensitive match and sort", () => {
  const items = [
    { id: 1, name: "banana" },
    { id: 2, name: "Apple" },
    { id: 3, name: "cherry" },
    { id: 4, name: "APRICOT" },
  ];
  assert.deepEqual(searchProducts(items, "A", 1, 10).map((p) => p.id), [2, 4, 1]);
});

test("page < 1 behaves as page 1; pageSize < 1 returns []", () => {
  assert.deepEqual(searchProducts(sample, "ap", 0, 1), searchProducts(sample, "ap", 1, 1));
  assert.deepEqual(searchProducts(sample, "ap", -5, 1), searchProducts(sample, "ap", 1, 1));
  assert.deepEqual(searchProducts(sample, "ap", 1, 0), []);
  assert.deepEqual(searchProducts(sample, "ap", 1, -3), []);
});

test("page beyond the last returns []", () => {
  assert.deepEqual(searchProducts(sample, "ap", 9, 10), []);
});

test("empty query matches everything; empty input returns []", () => {
  assert.equal(searchProducts(sample, "", 1, 10).length, 3);
  assert.deepEqual(searchProducts([], "a", 1, 10), []);
});

test("does not mutate the input", () => {
  const input = structuredClone(sample);
  const snapshot = structuredClone(sample);
  searchProducts(input, "", 1, 10);
  assert.deepEqual(input, snapshot);
});

test("handles 100,000 products quickly", () => {
  const big = Array.from({ length: 100_000 }, (_, i) => ({
    id: i,
    name: `Item-${(i * 7919) % 100_000}`,
  }));
  const t = performance.now();
  const out = searchProducts(big, "item", 3, 20);
  const ms = performance.now() - t;
  assert.equal(out.length, 20);
  assert.ok(ms < 1500, `took ${ms}ms`);
});
```

src/lib/findRoute.test.ts

```
import { test } from "node:test";
import assert from "node:assert/strict";
import { findRoute } from "./findRoute";

test("examples from the spec", () => {
```

```
  assert.equal(findRoute(["/products/:id", "/about"], "/products/42"), 0);
  assert.equal(findRoute(["/products/:id"], "/products/42/reviews"), -1);
  assert.equal(findRoute(["/about"], "/about/"), 0);
});

test("returns the FIRST matching pattern", () => {
  assert.equal(findRoute(["/a/:x", "/a/b"], "/a/b"), 0);
  assert.equal(findRoute(["/a/b", "/a/:x"], "/a/b"), 0);
  assert.equal(findRoute(["/a/b", "/a/:x"], "/a/c"), 1);
});

test("no partial segment matches", () => {
  assert.equal(findRoute(["/about"], "/aboutus"), -1);
  assert.equal(findRoute(["/products/:id"], "/product/42"), -1);
});

test("dynamic segment must be non-empty and match exactly one segment", () => {
  assert.equal(findRoute(["/p/:id"], "/p/"), -1);
  assert.equal(findRoute(["/p/:id"], "/p//"), -1);
  assert.equal(findRoute(["/p/:id"], "/p"), -1);
  assert.equal(findRoute(["/p/:id"], "/p/1/2"), -1);
});

test("trailing slash is ignored on patterns and paths; root works", () => {
  assert.equal(findRoute(["/about/"], "/about"), 0);
  assert.equal(findRoute(["/"], "/"), 0);
  assert.equal(findRoute(["/about"], "/"), -1);
});

test("only ONE trailing slash is ignored", () => {
  assert.equal(findRoute(["/about"], "/about//"), -1);
});

test("empty pattern list returns -1", () => {
  assert.equal(findRoute([], "/x"), -1);
});

test("scales to 10,000 patterns", () => {
  const patterns = Array.from({ length: 10_000 }, (_, i) => `/r${i}/:id/x`);
  assert.equal(findRoute(patterns, "/r9999/42/x"), 9999);
});
```

src/lib/buildBreadcrumbs.test.ts

```
import { test } from "node:test";
import assert from "node:assert/strict";
import { buildBreadcrumbs } from "./buildBreadcrumbs";

test("example 1", () => {
  assert.deepEqual(buildBreadcrumbs("/products/blue%20mug", { products: "Shop" }), [
    { label: "Shop", href: "/products" },
    { label: "blue mug", href: "/products/blue%20mug" },
  ]);
});

test("example 2: root has no breadcrumbs", () => {
  assert.deepEqual(buildBreadcrumbs("/", {}), []);
});

test("trailing and duplicate slashes are ignored", () => {
  assert.deepEqual(buildBreadcrumbs("/a//b/", {}), [
    { label: "a", href: "/a" },
    { label: "b", href: "/a/b" },
  ]);
});

test("hyphens become spaces only for unmapped segments", () => {
  assert.deepEqual(buildBreadcrumbs("/my-account/order-history", { "my-account": "Account" }), [
    { label: "Account", href: "/my-account" },
    { label: "order history", href: "/my-account/order-history" },
  ]);
});

test("labels are keyed by the DECODED segment", () => {
  assert.deepEqual(buildBreadcrumbs("/blue%20mug", { "blue mug": "Mug" }), [
    { label: "Mug", href: "/blue%20mug" },
  ]);
```

```
});

test("hrefs are encoded prefixes (unicode, reserved chars)", () => {
  const out = buildBreadcrumbs("/kategori/kopi%20%26%20teh/caf%C3%A9", {});
  assert.deepEqual(out.map((c) => c.href), [
    "/kategori",
    "/kategori/kopi%20%26%20teh",
    "/kategori/kopi%20%26%20teh/caf%C3%A9",
  ]);
  assert.equal(out[1]?.label, "kopi & teh");
  assert.equal(out[2]?.label, "café");
});

test("malformed percent-encoding does not throw", () => {
  const out = buildBreadcrumbs("/bad%E0%A4%A", {});
  assert.equal(out.length, 1);
  assert.equal(out[0]?.label, "bad%E0%A4%A");
});

test("prototype keys are not treated as labels", () => {
  const out = buildBreadcrumbs("/constructor/__proto__", {});
  assert.deepEqual(out.map((c) => c.label), ["constructor", "__proto__"]);
});
```

src/lib/formatIDR.test.ts

```
import { test } from "node:test";
import assert from "node:assert/strict";
import { formatIDR } from "./formatIDR";

test("formats IDR with dot thousands separator", () => {
  assert.equal(formatIDR(12500), "Rp 12.500");
  assert.equal(formatIDR(0), "Rp 0");
  assert.equal(formatIDR(999), "Rp 999");
  assert.equal(formatIDR(1234567), "Rp 1.234.567");
});

test("rounds to whole rupiah and handles edge values", () => {
  assert.equal(formatIDR(12500.6), "Rp 12.501");
  assert.equal(formatIDR(-1500), "-Rp 1.500");
  assert.equal(formatIDR(Number.NaN), "Rp -");
});
```

END OF TEST